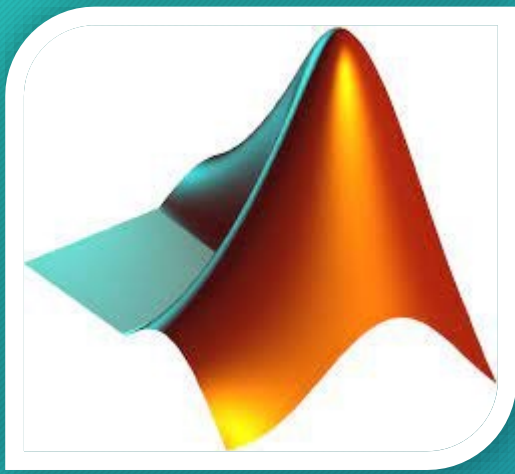




Communication Design

LAB.7

Wireless Communications
Multipath Channels

Ass.Lec.Ammar Alabbassi

Course Outline:

✓ 1- Simulate a Basic Digital Communications Link

- ▶ Implement the essential components of a single-carrier digital communications system, including additive white Gaussian noise.

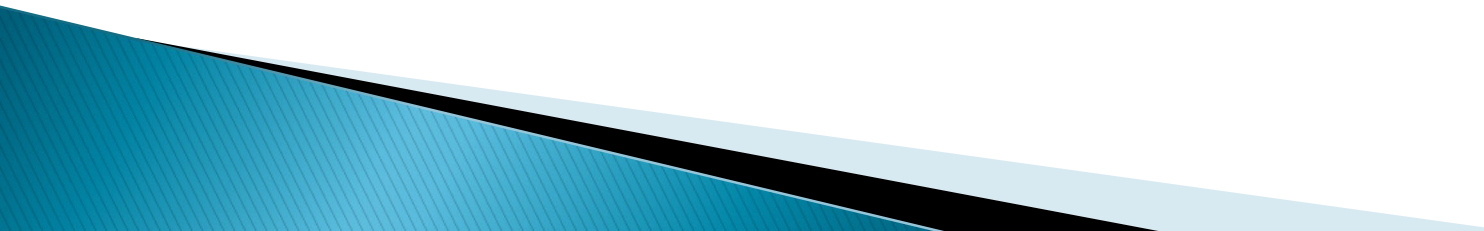
✓ 2- Pulse-Shaping Filters

- ▶ Incorporate transmit and receive filters into your simulation.

✓ 3- Multipath Channels

- ▶ Model a multipath channel and evaluate the impact on link quality.

4- OFDM

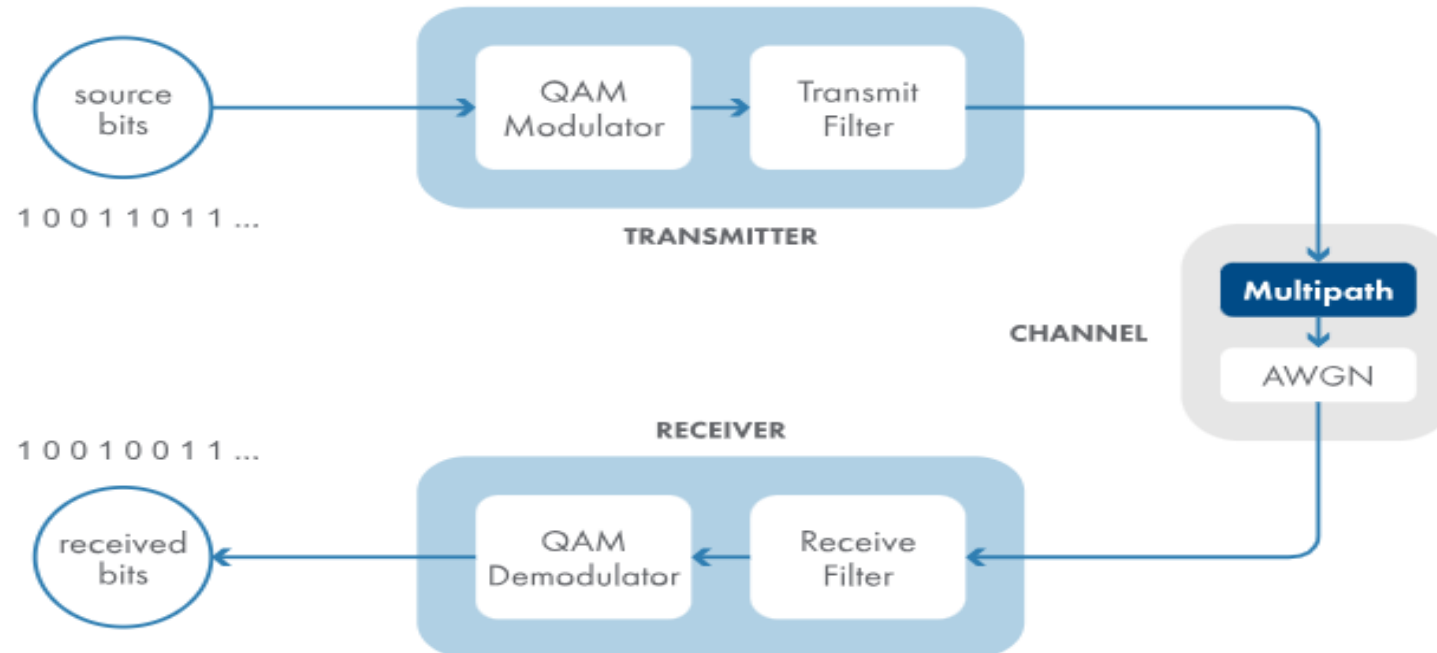
- ▶ Implement orthogonal frequency-division multiplexing (OFDM), a multicarrier modulation scheme.
- 

3- Multipath Channels

Model a Multipath Channel

In this activity, you'll add a multipath channel to your simulation. The multipath channel precedes the AWGN because the multipath is encountered in the air, before the signal reaches the receiver that adds the AWGN.

For simplicity, you'll simulate a static channel. However, in scenarios where the transmitter, receiver, or reflectors are moving, the channel impulse response is time varying.



3- Multipath Channels

Model a Multipath Channel



Task 1

This code sets up a 16-QAM link with filtering.

```
numBits = 20000;  
modOrder = 16; % for 16-QAM  
bitsPerSymbol = log2(modOrder); % modOrder = 2^bitsPerSymbol  
txFilt = comm.RaisedCosineTransmitFilter;  
rxFilt = comm.RaisedCosineReceiveFilter;  
  
srcBits = randi([0,1],numBits,1);  
modOut = qammod(srcBits,modOrder,"InputType","bit","UnitAveragePower",true);  
txFiltOut = txFilt(modOut);
```

Task 1

Background

- You can represent a multipath channel as a finite impulse response (FIR) filter. In MATLAB, an FIR filter can be defined by a vector of coefficients.
 $\mathbf{b} = [\mathbf{b0} \ \mathbf{b1} \ \dots \ \mathbf{bN}]$
- For a multipath channel, you can think of each nonzero coefficient as representing one path and each vector element (after the first) as a delay of one sample.
 $\mathbf{mp} = [\mathbf{p1} \ 0 \ 0 \ \mathbf{p2} \ 0 \ 0 \ \mathbf{p3}]$
- Since your signals are column vectors, it's good practice to be consistent and make the coefficient vector also a column vector. Transpose a row vector to make it a column vector.
 $\mathbf{mp} = [\mathbf{p1} \ 0 \ 0 \ \mathbf{p2} \ 0 \ 0 \ \mathbf{p3}]'$

TASK

Create a column vector named **mpChan** with 17 elements. Set the values of the 1st, 9th, and 17th elements to 0.8, -0.5, and 0.34, respectively. All other elements should be 0.

`spacing = zeros(7,1);`

`mpChan = [0.8; spacing; -0.5; spacing; 0.34]`

Task 2

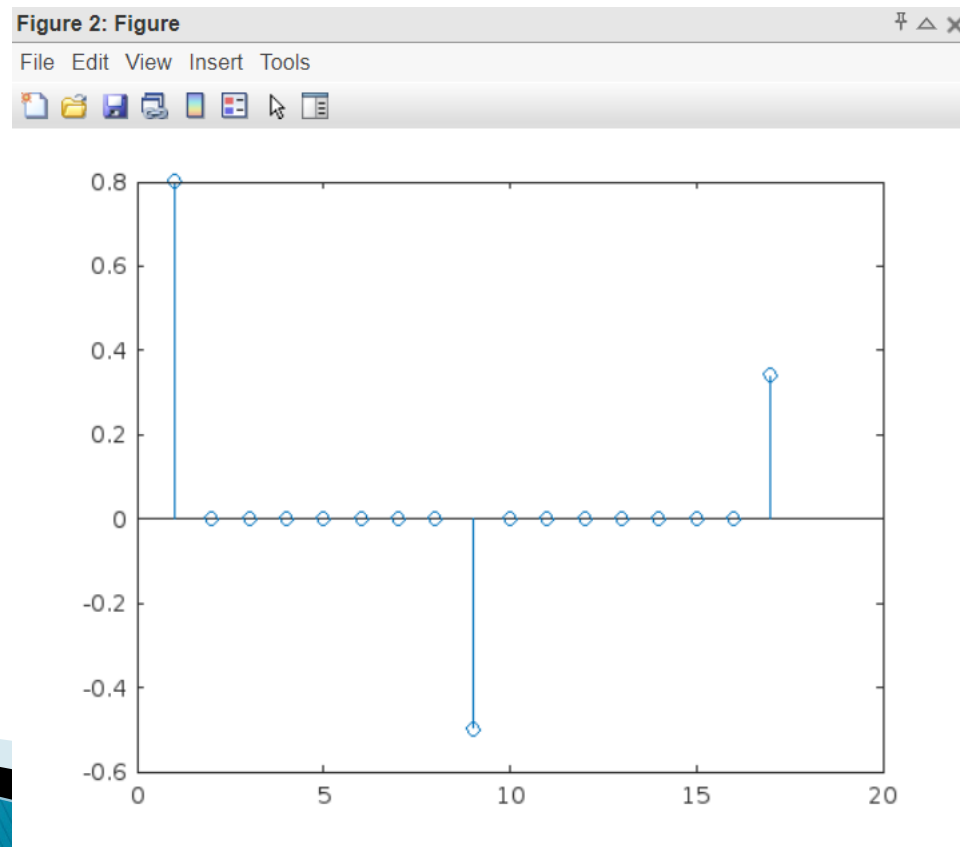
- ▶ You can visualize the impulse response of your multipath channel as a stem plot using the stem function.

`stem(mp)`

TASK

- ▶ Create a stem plot of the impulse response of the multipath channel **mpChan**.

`stem(mpChan)`



Task 3

- To apply the FIR filter to a signal, use the filter function. The first input is the impulse response, or the numerator coefficients of the filter. The second input is the denominator coefficients—for an FIR filter, this is always 1. The third input is the signal that you want to filter.

```
sigOut = filter(mp,1,sig)
```

TASK

- Apply the multipath channel mpChan to the transmitted signal txFiltOut. Name the output signal mpChanOut.
`mpChanOut = filter(mpChan,1,txFiltOut);`

Task 3

The following code is executed after the variable mpChanOut is created.

```
if exist("mpChanOut","var") % code runs after you complete Task 3
SNR = 15; % dB
chanOut = awgn(mpChanOut,SNR,"measured");
rxFiltOut = rxFilt(chanOut);
scatterplot(rxFiltOut)
title("Receive Filter Output")
demodOut = qamdemod(rxFiltOut,modOrder,"OutputType","bit","UnitAveragePower",true);
% Calculate the BER
delayInSymbols = txFilt.FilterSpanInSymbols/2 + rxFilt.FilterSpanInSymbols/2;
delayInBits = delayInSymbols * bitsPerSymbol;
srcAligned = srcBits(1:(end-delayInBits));
demodAligned = demodOut((delayInBits+1):end);
numBitErrors = nnz(srcAligned~=demodAligned)
BER = numBitErrors/length(srcAligned)
end
```

